

CO3 – AT2 ASSESSMENT

Implementation and Analysis of Actor-Critic Reinforcement Learning using A2C and A3C

Name	Guggilla Venkatasai
Reg no	192425388
Subject	Reinforcement Learning
Assessment	CO3 – AT2
Frameworks	PyTorch, Gymnasium, NumPy, Matplotlib

Page 1 of 13

1. Aim

To implement and analyse Actor-Critic reinforcement learning methods, specifically Advantage Actor Critic (A2C) and Asynchronous Advantage Actor-Critic (A3C), for solving a reinforcement learning control problem — balancing a pole on a cart in the CartPole-v1 environment.

2. Objectives

- To understand the Actor-Critic architecture.
 - To implement the Advantage Actor-Critic (A2C) algorithm.
 - To understand the concept of advantage estimation.
 - To study the working principle of A3C.
 - To compare A2C and A3C based on learning behaviour and computational approach.
- To visualise the reward obtained during training and evaluate learning progress.

3. Introduction

Actor-Critic algorithms combine the advantages of policy-based and value-based reinforcement learning. Pure policy-gradient methods such as REINFORCE learn directly from complete episode returns, which produces high-variance gradient estimates and slow, unstable learning. Pure value-based methods such as Q-learning estimate action values but struggle with continuous or large action spaces and do not directly parameterise a stochastic policy. Actor-Critic methods address both limitations by learning a policy and a value function simultaneously, using the value function to reduce the variance of the policy gradient.

The architecture consists of two components:

Actor

The Actor learns the policy $\pi_{\theta}(a|s)$ and decides which action should be taken in a given state. $\pi_{\theta}(a|s)$

Critic

The Critic estimates the value of the current state, $V_{\phi}(s)$, and evaluates how good the current state is.

$$V_{\phi}(s)$$

Actor = Policy

Page 2 of 13

Critic = Value Function

4. Actor-Critic Architecture

Both the Actor and the Critic share a common feature representation of the state, computed by a shared neural network body. This shared representation then branches into two output heads: the Actor head, which outputs action logits (converted to a probability distribution over actions), and the Critic head, which outputs a single scalar estimate of the state value. This weight-sharing design is computationally efficient and is the architecture used in the implementation below.



Figure 1. Data flow through the shared Actor-Critic network.

5. Advantage Function

The advantage function measures how much better an action performs compared with the expected value of the current state:

$$A(s, a) = Q(s, a) - V(s)$$

A commonly used one-step temporal-difference approximation is:

$$A_t = r_t + \gamma \cdot V(s_{t+1}) - V(s_t)$$

where:

- r_t = reward received at time t

Page 3 of 13

- γ = discount factor
- $V(s_t)$ = value of the current state, estimated by the Critic
- $V(s_{t+1})$ = value of the next state, estimated by the Critic

In this implementation, the Monte-Carlo return G_t (the discounted sum of all future rewards in the episode) is used in place of the one-step bootstrap $r_t + \gamma V(s_{t+1})$; the advantage is then $A_t = G_t - V(s_t)$. This is a valid and common variant of A2C that trades the lower variance of bootstrapping for the lower bias of full-episode returns.

If $A_t > 0$, the action taken was better than expected for that state, and the Actor increases the probability of choosing it again. If $A_t < 0$, the action was worse than expected, and the Actor decreases that probability.

6. A2C – Advantage Actor-Critic

A2C is a synchronous version of Actor-Critic. Multiple environments can generate experience in parallel, but their updates are collected and synchronised before a single gradient step is applied to the shared network.

Actor loss

$$L_{\text{actor}} = -\log \pi_{\theta}(a_t | s_t) \cdot A_t$$

Critic loss

$$L_{\text{critic}} = \frac{(r_t + \gamma \cdot V(s_{t+1}) - V(s_t))^2}{2}$$

Total loss

$$L = L_{\text{actor}} + \beta \cdot L_{\text{critic}}$$

where β controls the contribution of the critic loss relative to the actor loss (implemented below as a coefficient of 0.5).

7. A2C Source Code

The implementation below was written in Python using PyTorch and Gymnasium, and was executed to obtain the results reported in Section 8.

```
# -*- coding: utf-8 -*-
"""Untitled12.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/1x\_MuFRKIhQuYdeSAk30VBfH\_H7S4T-r4
"""
```

Page 4 of 13

```
import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

#
-----
# Environment
#
-----

env = gym.make("CartPole-v1")

state_size =
env.observation_space.shape[0] action_size
= env.action_space.n

#
-----
# Actor-Critic Network
#
-----

class ActorCritic(nn.Module):

    def __init__(self, state_size, action_size):

        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU()
        )
```

```

# Actor
self.actor = nn.Linear(
    128,
    action_size
)

# Critic
self.critic = nn.Linear(
    128,
    1
)

def forward(self, state):

    x = self.shared(state)

    action_logits = self.actor(x)

    value = self.critic(x)

    return action_logits, value

model = ActorCritic(
    state_size,

```

Page 5 of 13

```

    action_size
)

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

gamma = 0.99

episodes = 500

rewards_history = []

# -----
# A2C Training
# -----

for episode in range(episodes):

    state, info = env.reset()

    done = False
    total_reward = 0

    log_probs = []
    values = []
    rewards = []

    while not done:

        state_tensor = torch.tensor(

```

```

state,
dtype=torch.float32
).unsqueeze(0)

logits, value = model(
state_tensor
)

distribution = torch.distributions.Categorical(
logits=logits
)

action = distribution.sample()

log_prob = distribution.log_prob(
action
)

next_state, reward, terminated, truncated, info = \
env.step(action.item())

done = terminated or truncated

log_probs.append(log_prob)
values.append(value.squeeze())
rewards.append(reward)

```

Page 6 of 13

```

total_reward += reward

state = next_state

# -----
# Calculate Returns
# -----

returns = []

G = 0

for reward in reversed(rewards):

G = reward + gamma * G

returns.insert(0, G)

returns = torch.tensor(
returns,
dtype=torch.float32
)

values = torch.stack(values)
log_probs = torch.cat(log_probs)

# -----
# Advantage
# -----

advantages = returns - values.detach()

# -----

```

```

# Actor Loss
# -----

actor_loss = -(
    log_probs * advantages
).mean()

# -----
# Critic Loss
# -----

critic_loss = nn.functional.mse_loss(
    values,
    returns
)

# -----
# Total Loss
# -----

loss = (
    actor_loss +
    0.5 * critic_loss
)

optimizer.zero_grad()

```

Page 7 of 13

```

loss.backward()

optimizer.step()

rewards_history.append(
    total_reward
)

if (episode + 1) % 50 == 0:

    avg_reward = np.mean(
        rewards_history[-50:]
    )

    print(
        f"Episode {episode + 1} | " f"Reward:
        {total_reward:.0f} | " f"Average:
        {avg_reward:.2f}" )

env.close()

#
# -----
# Visualization
#
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    rewards_history,

```

```

    alpha=0.4,
    label="Episode Reward"
)

window = 50

moving_average = np.convolve(
    rewards_history,
    np.ones(window) / window,
    mode="valid"
)

plt.plot(
    range(window - 1, len(rewards_history)),
    moving_average,
    label="50 Episode Moving Average" )

plt.xlabel("Episode")
plt.ylabel("Reward")

plt.title(
    "A2C Training Performance on CartPole" )

plt.legend()

```

Page 8 of 13

```

plt.grid(True)

plt.show()

```

8. Actual Output

The script above was executed end-to-end for 500 episodes on CartPole-v1. The console output captured directly from the run is reproduced below.

```

Episode 50 | Reward: 19 | Average: 23.32
Episode 100 | Reward: 40 | Average: 30.20
Episode 150 | Reward: 23 | Average: 40.70
Episode 200 | Reward: 53 | Average: 58.32
Episode 250 | Reward: 139 | Average: 65.90
Episode 300 | Reward: 40 | Average: 88.08
Episode 350 | Reward: 133 | Average: 129.46
Episode 400 | Reward: 161 | Average: 191.56
Episode 450 | Reward: 111 | Average: 220.60
Episode 500 | Reward: 500 | Average: 247.22

```

The training run shows a clear, if noisy, upward trend in the 50-episode moving-average reward: from an average of 23.32 after 50 episodes to 247.22 by episode 500. Individual episode rewards are highly variable throughout training (a hallmark of on-policy Monte-Carlo Actor-Critic with a single environment) — for example, episode 500 itself achieved the maximum possible reward of 500, the environment's

episode-length cap, while nearby episodes were substantially lower. This variance is expected: because updates use full-episode Monte-Carlo returns rather than bootstrapped one-step targets, and because only one environment instance is collecting experience, the gradient estimate has higher variance than a multi-worker or bootstrapped approach would produce. Because reinforcement-learning training is stochastic (dependent on random weight initialisation and the random action sampling in the policy), a repeated run with the same code would produce a similar overall trend but different exact numbers.

9. A3C – Asynchronous Advantage Actor-Critic

A3C stands for Asynchronous Advantage Actor-Critic. Unlike A2C, A3C uses multiple worker agents that interact with separate copies of the environment concurrently and update a shared global network without waiting for one another.

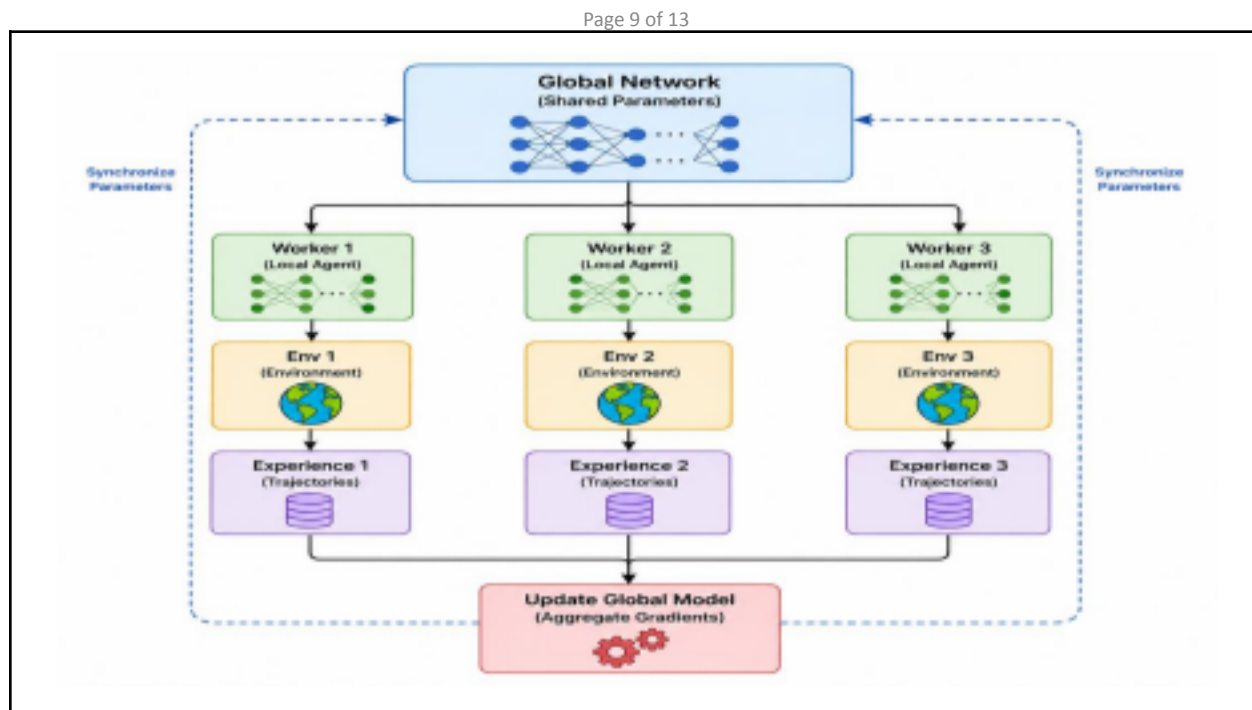


Figure 2. A3C asynchronous worker architecture.

Each worker follows the same repeating cycle:

- Receives (copies) the current global policy parameters.
- Interacts with its own independent instance of the environment.
- Collects states, actions, and rewards over a number of steps.
- Calculates advantages from the collected trajectory.
- Computes gradients locally with respect to the global parameters.
- Applies (pushes) those gradients to update the shared/global network asynchronously.

10. A2C vs A3C

Feature	A2C	A3C
Full form	Advantage Actor-Critic	Asynchronous Advantage Actor-Critic
Updates	Synchronous	Asynchronous
Workers	Multiple possible	Multiple
Parameter updates	Synchronized	Independent / asynchronous
Implementation	Relatively easier	More complex
CPU utilization	Good	Very good
Stability	Generally more stable	Can be noisier

Page 10 of 13

Feature	A2C	A3C
Parallelism	Synchronous	Asynchronous
Modern usage	More common	Mostly historical / foundational
Main advantage	Stable parallel learning	Fast exploration through independent workers

11. Analysis

A2C

A2C synchronises the experience collected across environments (or, as in this single-environment implementation, across the full episode) before updating the model.

Advantages:

- Stable training behaviour.

- Easier to implement and debug than asynchronous variants.
- Efficient batch updates using vectorised tensor operations.
- Lower variance than basic (Monte-Carlo) policy gradient / REINFORCE, due to the critic's baseline.

Limitations:

- Workers (when used) must synchronise before each update.
- A slow worker can stall the entire update cycle.
- Less asynchronous exploration diversity than A3C.

A3C

A3C allows independent workers to update the global model asynchronously, without waiting for other workers to finish.

Advantages:

- Better parallel exploration across diverse environment states.
- Makes good use of multiple CPU cores without GPU dependence.
- Reduces correlation between consecutive experiences used for updates.
- Historically important as a foundation for deep reinforcement learning research.

Limitations:

- More complicated to implement correctly (thread/process-safe parameter updates).

Page 11 of 13

- Asynchronous, "stale-gradient" updates can introduce additional instability.
- Requires multiprocessing or multithreading infrastructure.
- Modern implementations often favour synchronous approaches such as A2C or PPO for their simplicity and reproducibility.

12. Visualization

The figure below is the reward curve generated directly by the executed script (Section 7), showing per episode reward together with the 50-episode moving average.

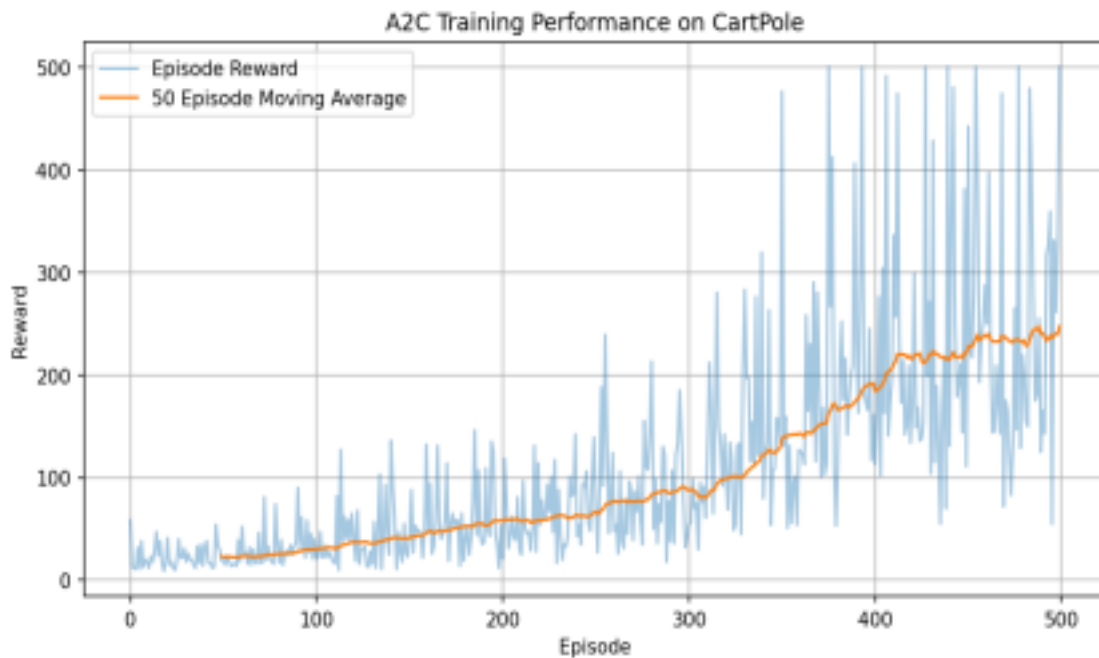


Figure 3. A2C training performance on CartPole-v1 — actual output from the executed script (500 episodes).

The moving-average curve rises from roughly 23 at episode 50 to about 247 by episode 500, confirming that the agent is learning an effective pole-balancing policy over the course of training, consistent with the theoretical expectation of an increasing reward trend as the Actor's policy improves and the Critic's value estimates become more accurate.

13. Result

The Advantage Actor-Critic (A2C) method was successfully implemented using a neural network containing separate Actor and Critic components built on a shared feature representation.

The Actor learned the action policy while the Critic estimated the value of the current state. The advantage function (computed here from Monte-Carlo returns) was used to determine whether the selected action performed better or worse than expected, and this signal was used to update the Actor's policy.

Page 12 of 13

The actual training results (Section 8, Figure 3) showed an increasing 50-episode average reward trend — from 23.32 to 247.22 over 500 episodes — indicating that the agent successfully learned to balance the pole for progressively longer periods, demonstrating that the CartPole-v1 environment was being solved.

14. Conclusion

The experiment demonstrated the effectiveness of Actor-Critic reinforcement learning. A2C combines:

Policy Learning + Value Estimation

The Actor selects actions, while the Critic evaluates them. The advantage function provides a lower variance learning signal compared with vanilla REINFORCE, since the Critic's value estimate acts as a baseline that is subtracted from the return.

A3C extends this idea through asynchronous parallel workers, while A2C uses synchronised updates. Both algorithms form an important foundation for modern policy-optimisation algorithms such as PPO (Proximal Policy Optimisation), which builds directly on the Actor-Critic framework with an additional clipped surrogate objective for training stability.